GRC Guide to GitHub

Governance-as-Code. Audit Evidence on Demand.
The Sovereign Source of Truth.

What's Inside

- *Core GitHub concepts*
- *Governance-as-Code in practice*
- *GitHub as a Audit Evidence Engine*
- *Real-World GRC Scenarios*
- *Framework Mapping & Commands*

Kunal RK

Contents

01 What is GitHub

02 How GitHub Became the Centre of Audit Evidence

03 Core Concepts Through a GRC Lens

04 GitHub as the Ultimate Audit Evidence Engine

05 Governance-as-Code: The Philosophy in Practice

06 Real-World GRC Scenarios

07 Framework Mapping

08 The GRC Professional's GitHub Audit Playbook

09 GitHub and AI Governance

10 Command Vault: Every GitHub Command You Need

11 Glossary of GitHub Terms for GRC

SECTION 01

What is GitHub

GitHub is a website. A website where work gets saved, and that saved work lives on the internet so other people can go through it, see every change ever made, and verify that the work is exactly what it claims to be. That is the whole thing. Everything else in this guide is detail built on top of that.

Think of it as Google Drive with one critical difference. In Google Drive, someone can make a quiet edit and save it. The previous version is gone. In GitHub, every change is stamped with who made it, when, and exactly what changed. Nothing overwrites. The record is permanent.

The Filing Cabinet That Never Lies

Every time a document goes in, changes, or comes out, the cabinet takes a photograph of itself and stamps it with the name of whoever touched it and the exact time. Nobody can alter a past photograph. Nobody can claim a document was always there if it was not.

That filing cabinet is a GitHub repository. The photograph is a commit. The person who approves a new document entering the cabinet is the pull request reviewer. For GRC, this is not a metaphor. This is audit evidence.

GRC Application

- GitHub is free. Any GRC professional can create a repository and start storing version-controlled governance documents today. No IT request required.
- A policy document in GitHub with a full commit history is stronger audit evidence than the same document in a shared drive folder with no version control.
- Almost every engineering team already uses GitHub. The question is whether the GRC function can read what is already there.

SECTION 02

How GitHub Can Become the Centre of Audit Evidence

For years, GRC operated at a comfortable distance from engineering. Policies were written. Audits were run. Controls were mapped. When someone mentioned GitHub, the meeting moved on. That distance has a cost, and it is becoming visible.

The systems an organisation relies on, the ones storing customer data, running AI models, and serving regulated workloads, are built and changed through platforms like GitHub. Every configuration change, every access policy update, every infrastructure decision leaves a trace there. If that trace cannot be read, the audit cannot reach what matters.

> **Paper compliance captures what was intended. GitHub captures what actually happened.**

What This Looks Like Across GRC Domains

The gap between GRC knowledge and GitHub knowledge shows up differently by domain, but the outcome is always the same: weaker evidence, a reduced ability to assess what is actually in place, and a harder conversation with the auditor.

Audit

Change control evidence arrives as screenshots and email chains. A cryptographically signed, timestamped change history exists in GitHub. The screenshot proves nothing about the actual system state.

Risk Assessment

A risk is scored as managed because a policy says the control is in place. The policy is 18 months old. The system it describes was rewritten three months ago. The commit history shows that immediately.

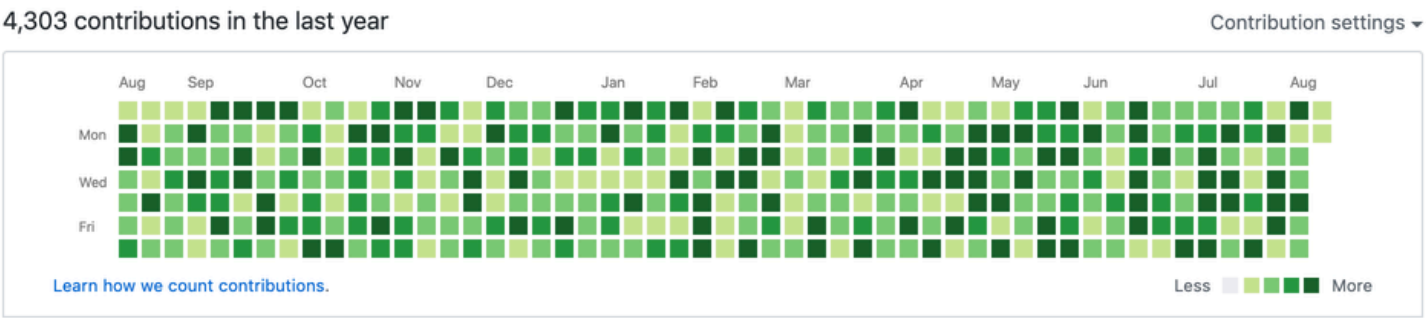
TPRM

A vendor confirms they have a change management process. The questionnaire is accepted. Nobody checked whether branch protection rules exist, whether direct commits to production are permitted, or whether self-approval is possible.

Incident Response

A breach requires a forensic timeline. Reconstructing it means knowing exactly what changed, when, and who approved it. Without the ability to read a commit history, the investigation stalls at the point where GitHub evidence begins.

> **The Standard This Guide Sets**
>
> Not developer knowledge. GRC knowledge: open a repository, read a commit history, understand what a pull request approval means as audit evidence, and identify when access controls or change workflows have gaps. That is the capability this guide builds.



SECTION 03

Core Concepts Through a GRC Lens

Every GitHub concept maps to something a GRC professional already understands. Each one is explained plainly below, then applied directly to audit, risk, and compliance work.

Repository

A container for a project. Everything related to it lives inside: the files, the full change history, the approvals, the discussions. One repository per audit scope item.

GRC Application

- One repository per control domain, vendor, or regulated system. Audit evidence, policies, and change history in one place.
- Private repositories require explicit access grants, creating a natural access control record.

Framework Link

ISO 27001 A.8, SOC 2 CC6, NIST CSF PR.AC: organised, version-controlled, auditable documentation satisfies evidence requirements without additional process overhead.

Commit

A saved change. Every commit has a unique cryptographic fingerprint, an author, a timestamp, and a message. Past commits cannot be modified without leaving a visible trace.

GRC Application

- When was a policy last reviewed and by whom? The commit history answers that.
- This is not approximate evidence. It is cryptographically verifiable evidence.

Framework Link

ISO 27001 A.12.4, SOC 2 CC7, NIST CSF DE.CM: commit history is a native audit trail. No additional tooling required.

Branch

A parallel version of a repository for proposed changes. The original version, main, remains untouched until the branch is reviewed and approved.

GRC Application

- Branching is change management built into the platform. Changes are proposed, reviewed, approved or rejected. The approval is recorded permanently.
- Branch protection rules enforce that no one can bypass this process, even with repository access.

Framework Link

ISO 27001 A.12.1.2, SOC 2 CC8, NIST CSF PR.IP-3: change management evidence native to the platform. The process is enforced, not just documented.

Pull Request

A formal proposal to merge branch changes into main. Reviewers examine, comment, approve or reject. The entire discussion is permanently recorded.

GRC Application

- A four-eyes approval record with full commentary and a tamper-proof decision log.
- A pull request approval is stronger change management evidence than a change advisory board email. It is linked to the exact change, not a description of it.

Framework Link

SOC 2 CC8.1, ISO 27001 A.12.1.2, PCI DSS Req 6.4: pull request records directly satisfy the approval evidence requirement.

Hash (SHA)

A cryptographic fingerprint generated from the exact content of every commit. If even one character changes, the hash changes completely.

GRC Application

- If someone claims a document was always version X and the hash does not match, the claim does not hold.
- For forensic investigations, hashes establish the exact state of any file at any point in history with mathematical certainty.

Framework Link

NIST SP 800-53 SI-7, ISO 27001 A.10.1: the SHA hash is a native integrity control. No configuration required. It applies to every commit automatically.

Issue

A formal record of a task, problem, or finding within a repository. Assignable, labelled, linked to commits and pull requests, and closed with documented resolution.

GRC Application

- An audit finding becomes a GitHub issue: assigned an owner, prioritised, every update recorded. Closure links to the commit or PR that resolved it.
- Remediation tracking that proves remediation. Not a spreadsheet with a closed column.

Framework Link

ISO 27001 A.16, SOC 2 CC7.4, NIST CSF RS.MI: issues with linked commits provide traceable remediation evidence that point-in-time documents cannot match.

SECTION 04

GitHub as the Ultimate Audit Evidence Engine

Most audit evidence is collected after the fact. A request arrives and the team goes searching for the email, the ticket, the meeting note. Sometimes it exists. Sometimes it does not. Sometimes it exists but cannot be verified as genuine.

GitHub works differently. The evidence is created at the moment the work happens, stored automatically, permanently, and in a form that cannot be quietly edited.

The Four Questions Every Audit Asks

Every audit, regardless of framework, resolves to four questions. GitHub answers all four without anyone having to reconstruct a timeline.

Question	How GitHub Answers It
WHO	Every commit, PR, and merge is tied to an authenticated GitHub account. Identity is verified. Cannot be falsified without account compromise.
WHAT	GitHub stores a line-by-line diff for every change ever made. Not a summary. The exact character-level difference between before and after.
WHEN	Every action carries an immutable UTC timestamp from GitHub servers, not the user machine. It cannot be altered retroactively.
WHY	Commit messages and PR descriptions capture the rationale for every change. This is where "updated retention period to align with GDPR Article 17" lives permanently.

Evidence Quality: GitHub vs Traditional Methods

- Email approval vs pull request: the PR is immutable, cryptographically linked to the exact change, and reviewer identity is verified. The email is not.
- SharePoint document vs commit plus merge: the commit is automatic, the review workflow is enforced, and the approval is recorded permanently.
- Spreadsheet audit log vs commit history: the commit history is platform-generated, SHA-verified, and impossible to alter without detection.

SECTION 05

Governance-as-Code: The Philosophy in Practice

Governance-as-Code means treating governance artefacts the way engineers treat software: version-controlled, reviewed before changes go live, deployed with a full audit trail. Policies. Risk registers. Control mappings. All of it.

The alternative is paper compliance: a Word document emailed to an approver, saved to a shared drive with no version history, updated inconsistently, with no way to prove what was in force on any given date. Regulators are increasingly aware of the gap.

Governance Artefact	As Governance-as-Code in GitHub
Policy document	Stored in a repository. Changes go through pull requests with mandatory reviewers. The last commit date is the last review date. The merge approver is the reviewer.
Risk register	Every entry change has a commit. New risks, updated scores, linked controls: all recorded. The register as it existed on any date is retrievable in seconds.
Control evidence	GitHub Actions runs compliance checks automatically and logs results. The control is not just documented. It runs, and the execution is evidenced.
Vendor assessment	A standard questionnaire stored as a versioned file. You know which version was sent to which vendor and when.
Incident log	Records stored in the repository with timestamps. Linked to any commits that addressed the issue. Remediation is traceable, not asserted.

The Shift in Thinking

Stop treating governance documents as files to manage. Treat them as code to deploy. Code has owners, reviewers, and a deployment history. It cannot be quietly edited without leaving a trace. Governance documentation held to that standard is not a liability in an audit. It is an asset.

SECTION 06

Real-World GRC Scenarios

Two situations that arise in real audit and compliance engagements. Each one shows what changes when a GRC professional can read GitHub evidence versus when they cannot.

Scenario 1: The Auditor Asks for Change Control Evidence

An SOC 2 Type II audit is underway. The auditor requests evidence that all production changes went through an approved process: peer review before deployment, separation of proposer from approver, and a record of every change in the audit period.

Without GitHub Knowledge

The team collects email chains, change advisory board minutes, and a manually maintained change log. The auditor notes the spreadsheet was last updated six weeks ago and three email chains have no clear approval decision. A finding is raised.

With GitHub Knowledge

The pull request log for the audit period is exported. Branch protection rules confirm no direct commits to main. Required reviewer configuration confirms the proposer cannot self-approve. Every change is timestamped with reviewer identity and approval decision. The evidence request closes in one session.

- ISO 27001 A.12.1.2, SOC 2 CC8.1, PCI DSS Req 6.4

Scenario 2: Verifying a Vendor's Change Management Claims

During a TPRM review, a vendor asserts all changes to systems processing customer data go through a formal change management process with mandatory peer review. A completed questionnaire is submitted.

Without GitHub Knowledge

The questionnaire is accepted alongside a policy document describing the change management process. The control is marked as evidenced.

With GitHub Knowledge

The assessment requests read access to a sample of the vendor's repositories, or at minimum an export of pull request logs for the last quarter. Branch protection rules are checked. Direct commits to main are looked for. A policy document describes intent. The repository configuration proves implementation. These are different things.

- ISO 27001 A.15.1, SOC 2 CC9.2, NIST CSF ID.SC-4

> **A policy document describes intent. The repository configuration proves implementation. These are different things.**

SECTION 07

Framework Mapping

GitHub features map directly to specific framework controls when configured deliberately. Use this table when scoping audits, designing controls, or making the business case for adopting GitHub in a governance programme.

Framework	Control / Clause	GitHub Feature
ISO 27001	A.12.1.2 Change Management	Pull Requests, Branch Protection, Required Reviewers
ISO 27001	A.12.4.1 Event Logging	Commit History, Audit Log, GitHub Actions Log
ISO 27001	A.9.1.2 Access Control	Repository Permissions, Team Access, SAML SSO
ISO 27001	A.6.1.2 Segregation of Duties	Required Reviewers, CODEOWNERS, Branch Protection
ISO 27001	A.5.1.2 Policy Review	Commit History, Pull Request Approvals, Version Tags
ISO 27001	A.14.2.2 Secure Development	CI/CD Scans, Dependabot, Secret Scanning
SOC 2	CC8.1 Change Management	Pull Requests, Approval Workflows, Deployment Logs
SOC 2	CC6.1 Logical Access	Repository Access, Team Permissions, Access Reviews
SOC 2	CC7.2 System Monitoring	GitHub Actions, Audit Log, Security Alerts
SOC 2	CC9.2 Vendor Management	Repository Visibility, Branch Protection Evidence
NIST CSF	PR.AC-4 Access Permissions	Role-Based Repository Access, Team Hierarchies
NIST CSF	PR.IP-3 Config Management	Infrastructure-as-Code Repos, Change History
NIST CSF	RS.AN-3 Forensics	Git Blame, Commit Diff, Audit Log Export
EU AI Act	Art. 12 Record Keeping	Model Card Versioning, Training Data Changelogs
EU AI Act	Art. 17 Quality Management	CI/CD Testing Pipelines, Automated Validation

Framework	Control / Clause	GitHub Feature
ISO 42001	A.6.2 AI System Lifecycle	Model Repository, Version Tags, Release History
PCI DSS	6.4 Change Control	PR Approvals, Production Branch Protection, Audit Trail
GDPR	Art. 5(2) Accountability	Version-Controlled Policies, Commit-Evidenced Reviews

SECTION 08

The GRC Professional's GitHub Audit Playbook

When reviewing an organisation's use of GitHub, these are directives. Request this. Verify this. Flag this.

Branch Protection Configuration

- **REQUEST:** Branch protection rules for the main branch of each in-scope repository.
- **VERIFY:** Direct commits to main are disabled. All changes must arrive via pull request.
- **VERIFY:** At least one required reviewer is configured and the author cannot self-approve.
- **VERIFY:** Automated status checks are required to pass before merging.
- **FLAG:** Any repository with no branch protection on main. No change management control exists.
- **FLAG:** Any repository where the required reviewer count is zero or self-approval is permitted.
- **FRAMEWORK:** ISO 27001 A.12.1.2, SOC 2 CC8.1, PCI DSS Req 6.4.2

Access Control Review

- **REQUEST:** All users with write or admin access to production repositories, including when access was granted and by whom.
- **VERIFY:** Access follows least privilege. Developers have access only to relevant repositories.
- **VERIFY:** Admin access is restricted to a small named set of individuals with documented justification.
- **FLAG:** Any accounts with admin access that have not committed in the last 90 days. These are stale privileged accounts.
- **FLAG:** Service accounts or bot accounts with write access. These require scoped tokens, not broad permissions.
- **FRAMEWORK:** ISO 27001 A.9.2, SOC 2 CC6.1, NIST CSF PR.AC-4

Secrets and Vulnerability Management

•**REQUEST:** Confirmation that secret scanning and Dependabot are enabled across all repositories.

• **VERIFY:** Push protection is set to Block, not Warn. Commits containing secrets should be blocked before they reach the repository.
• **VERIFY:** Critical dependency vulnerability alerts have remediation timelines aligned with policy.
• **FLAG:** Any open secret scanning alert. A committed secret is a compromised secret until proven otherwise.
• **FLAG:** Critical Dependabot alerts older than 30 days with no linked issue or PR showing active remediation.
• **FRAMEWORK:** ISO 27001 A.10.1, A.12.6.1, SOC 2 CC6.1, NIST CSF ID.RA-1

> **Every repository with no branch protection on main is a change management failure waiting to be found by an auditor.**

SECTION 09

GitHub and AI Governance

The EU AI Act, NIST AI RMF, and ISO 42001 all require a documented, traceable, ongoing record of how AI systems are built, changed, monitored, and governed. GitHub is where that record naturally lives.

Every AI system has a codebase. That codebase lives in a repository. Model versions, training data references, deployment configurations, monitoring thresholds: all in version control. The question is whether the GRC function can read it.

Governance Artefact	How It Appears in GitHub
Model Card	Purpose, training data, performance metrics, limitations. Versioned. Every update is a pull request with a named approver.
Training Data Changelog	What data was used, when it was updated, who approved each update. Version-controlled alongside the model.
Bias and Fairness Assessment	Test results stored as files. Model changes trigger automated re-testing via CI/CD. Results linked to the release.
Deployment Configuration	How and where the model deploys. Branch protection ensures no deployment without an approved pull request.
Risk Register Entry	Identified risks, versioned. Changes reviewed through pull requests. Every update carries author and rationale.
Incident Log	AI-related incidents stored with timestamps and responsible parties. Linked to commits that addressed the issue.

EU AI Act and ISO 42001: What GitHub Satisfies

- EU AI Act Art. 11 (Technical Documentation): model cards and system design documents as versioned files with full approval history.
- EU AI Act Art. 12 (Record Keeping): immutable commit history as the audit trail for AI system development and every subsequent change.
- EU AI Act Art. 17 (Quality Management): CI/CD pipelines enforcing testing requirements before every deployment. The control is enforced, not just stated.
- ISO 42001 Cl. 7.5 (Documented Information): version-controlled policies and processes with commit-evidenced review dates and named approvers.
- ISO 42001 Cl. 9.1 (Monitoring and Evaluation): automated checks via GitHub Actions running on schedule with logged, auditable results.

SECTION 10

Command Vault: Every GitHub Command You Need

A command is an instruction typed to make GitHub do something. The tables below give the exact instructions GRC professionals need, plain-English explanations of what each does, and why it matters for audit work.

Before You Start

Git must be installed. On Windows: install Git for Windows from git-scm.com (provides Git Bash). On Mac or Linux: open Terminal. Once installed, every command below works identically on every operating system.

Getting Started and Audit Trail Commands

Command	What It Does	GRC Use Case
`git clone [URL]`	Downloads a full copy of a repository including complete history	Starting point for any evidence review of a repository in scope
`git log`	Shows the full commit history: author, timestamp, and message for every change	Primary audit trail command. The complete change record for a repository
`git log --oneline`	Compact single-line view of the commit timeline	Quick scan for unusual activity or gaps in commit frequency
`git log --author="[name]"`	Filters commit history to one named person only	Identify all changes by a specific individual during an investigation
`git log --since="YYYY-MM-DD" --until="YYYY-MM-DD"`	Shows all commits within a specific date range	Scope evidence collection exactly to the audit period

Command	What It Does	GRC Use Case
`git log -- [filename]`	Complete change history for one specific file	Track every change ever made to a policy document or control record
`git diff [commit1] [commit2]`	Line-by-line differences between two specific commits	Prove exactly what changed between two versions. Essential for change management evidence
`git show [hash]`	Full details of one commit: who, when, message, and all changes	Examine any individual change in detail during an investigation
`git blame [filename]`	Shows who last modified each line of a file and in which commit	Attribute every line of a document or configuration to a person and timestamp
`git log --merges`	Shows only merge commits, each representing an approved pull request	Verify changes went through approval before reaching production
`git rev-parse HEAD`	Shows the SHA hash of the most recent commit	Record the current state fingerprint of a repository for integrity evidence
`git tag [version] -m "[note]"`	Creates a named permanent marker on a specific commit	Tag policy versions (e.g. v2.1-approved-2024-Q1) as permanent reference points

Branch, Document and Policy Commands

Command	What It Does	GRC Use Case
`git branch -a`	Lists all branches including remote ones on the GitHub server	Full visibility of active branches. Identify unreviewed long-running branches
`git log main..[branch]`	Shows commits on a branch not yet merged to main	See what changes are pending approval on any open branch
`git commit -m "[message]"`	Saves staged changes with a permanent description	The message is permanent audit evidence. Write it precisely: what changed and why

Command	What It Does	GRC Use Case
`git push origin [branch]`	Sends local commits to GitHub for review and approval	Publish changes to trigger the review and approval workflow
`git pull`	Downloads the latest changes from GitHub to the local copy	Always run before starting work to confirm the current version is being edited

SECTION 11

Glossary of GitHub Terms for GRC

Every term defined through the lens GRC professionals encounter it.

Term	GRC-Relevant Definition
Actions	GitHub's automation engine. Runs workflows on triggers: commits, pull requests, schedules. Used to embed compliance checks directly into the development workflow.
Audit Log	Enterprise GitHub feature. Records every administrative action: access grants, permission changes, repository creation. Exportable, immutable, timestamped.
Branch	A parallel version of a repository for proposed changes. Changes do not affect main until reviewed and approved. The technical mechanism for change management control.
Branch Protection	Rules applied to a branch: require pull request review, prevent direct commits, mandate status checks. The enforcement layer for change management policy.
CODEOWNERS	A configuration file that automatically assigns required reviewers to changes affecting specific files or directories.
Commit	A saved, recorded change. Every commit has a unique SHA hash, an author, a timestamp, and a message. The atomic unit of audit evidence in GitHub.
Dependabot	GitHub's automated dependency vulnerability scanner. Raises alerts and creates fix pull requests when known vulnerabilities are found in dependencies.
Diff	A line-by-line comparison showing exactly what changed between two file versions. The "what was changed" evidence in any change management review.
Fork	An independent copy of an entire repository. Creates supply chain risk and licence obligations that must be assessed in TPRM and SBOM contexts.
Hash (SHA)	A cryptographic fingerprint unique to the exact content of a commit. Any change produces a completely different hash. Tamper-evidence built into every commit.
Issue	A formal record of a task, finding, or risk. Assignable, linkable to commits and pull requests. Functions as an audit-trail-enabled action tracker.

Term	GRC-Relevant Definition
Main	The primary, authoritative branch. Should always be protected by branch protection rules.
Merge	Incorporating approved branch changes into another branch. Requires all required reviews to be approved first.
PR (Pull Request)	A formal proposal to merge changes. Contains the diff, review discussion, approval decisions, and merge record. The primary change management evidence artefact.
Release	A named, tagged version at a specific point in time. In AI governance, releases correspond to model versions and deployment events.
Repository	A project container on GitHub. Holds all files, history, branches, issues, and pull requests. The evidence package for an audit scope item.
Secret Scanning	Scans committed content for credentials and tokens. Push Protection blocks commits containing secrets before they reach the repository.
Tag	A permanent named marker on a specific commit. Useful for policy versions, release versions, and audit period end states.
Workflow	A GitHub Actions file stored in .github/workflows. Defines when automation runs and what it does, including compliance checks and evidence generation.